

CSCE 735 PARALLEL COMPUTING · MODULE 08 · LECTURE NOTES

GPU Programming

Thousands of threads, and a machine that only makes sense once.

MODULE OUTCOMES

Explain the CUDA execution model and map it to the hardware.

Explain warps and why divergence costs.

Explain the memory hierarchy and coalescing.

Explain occupancy and what it is for.

Write a correct kernel and launch it.

1 The execution model

Software abstraction	Hardware reality
Thread	One lane of a SIMD unit.
Warp (32 threads)	The real unit of execution. Executed in lockstep, issuing one instruction for all 32 lanes. The hardware schedules warps; it does not schedule threads.
Block (up to 1024 threads)	Resident on one streaming multiprocessor for its entire lifetime, sharing that SM's shared memory and able to synchronise with <code>__syncthreads()</code> .
Grid (many blocks)	Distributed across all SMs. Blocks must be independent — they may execute in any order, and any subset may be resident simultaneously.

Block independence is the design decision that makes CUDA portable. Because blocks cannot communicate or depend on execution order, the same binary runs correctly on a device with four SMs and on one with a hundred and twenty-eight — the runtime simply schedules more or fewer blocks concurrently. Permitting inter-block dependencies would have tied every program to a particular device's resources.

THE WARP IS THE REAL UNIT, NOT THE THREAD

Thirty-two threads execute in lockstep, issuing the same instruction on the same cycle across 32 lanes. This is SIMD hardware with a thread-shaped programming model layered over it.

So a block of 100 threads occupies four warps, the fourth of which has 28 of its 32 lanes idle for the entire kernel. **Always use block sizes that are multiples of 32**, and 128 or 256 is the usual starting point.

And a branch that goes different ways within a warp serialises — see below.

Essentially everything about GPU performance follows from this. Divergence, coalescing, and occupancy are all direct consequences of 32 lanes executing together, and a model that treats GPU threads as independent will mispredict performance every time.

DIVERGENCE: BOTH BRANCHES EXECUTE

If some lanes of a warp take the if branch and others take the else, the warp executes both — first one with the other lanes masked off, then the other with the first set masked off.

So the cost is the sum of the two branches, not the maximum. A two-way divergent branch costs twice what a uniform one would, and a switch with eight cases taken differently across a warp costs eight times.

Divergence between warps is entirely free. If warp 0 takes one path and warp 1 takes the other, each executes only its own path — they are scheduled independently and never interact.

So organise data so that threads within a warp take the same path. Sorting the input by the branch condition, padding, or restructuring the decomposition so that divergence falls on warp boundaries is frequently a large win — and it is a data layout decision rather than a code change.

2 Memory

Space	Scope	Latency	Notes
Registers	One thread	~1 cycle	Plentiful, and register usage per thread limits occupancy (§3).
Shared memory	One block	~20 cycles	A programmer-managed cache. Fast, small (tens of KB per SM), and explicitly loaded. Using it well is most of GPU optimisation (Module 09).
L1 / L2 cache	SM / whole device	~30 / ~200 cycles	Automatic. L1 shares hardware with shared memory on most architectures.
Global memory	Whole device	400–800 cycles	Large (gigabytes), slow, and coalescing determines whether you get its bandwidth.
Constant memory	Device, read-only	Cached	Broadcasts cheaply when a whole warp reads the same address — ideal for kernel parameters and lookup tables.

Space	Scope	Latency	Notes
Local memory	One thread	Global memory speed	Despite the name, it lives in global memory. Used for register spills and for arrays indexed dynamically. A silent and severe performance bug — check the compiler's register and spill report.

COALESCING IS THE MOST IMPORTANT MEMORY RULE

The memory system serves a warp's 32 accesses as a small number of wide transactions, typically 32 or 128 bytes each.

If the 32 threads read 32 consecutive floats, that is a single 128-byte transaction. One request, full bandwidth, nothing wasted.

If they read 32 scattered addresses, that is up to 32 separate transactions — and each transaction still fetches a full cache line, so **the memory traffic can be 32 times larger** for exactly the same useful data.

So thread i should access element i , and data should be laid out structure-of-arrays rather than array-of-structures. **This is the same rule as Module 02's vectorisation guidance,** with a considerably steeper penalty for violating it — on a CPU a strided access is slower, and on a GPU it can cost an order of magnitude.

```
// COALESCED: consecutive threads, consecutive addresses
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) c[i] = a[i] + b[i];

// NOT COALESCED: stride between consecutive threads
if (i*stride < n) c[i] = a[i * stride];

// NOT COALESCED: array of structures -- each thread wants one field,
// so the warp reads 32 widely separated words
struct P { float x, y, z, w; };
out[i] = p[i].x;           // reads 1 float of every 4
// FIX: separate arrays for x, y, z, w
```

3 Occupancy

OCCUPANCY EXISTS TO HIDE MEMORY LATENCY

Global memory latency is 400 to 800 cycles. A CPU hides this with large caches, prefetching, and out-of-order execution — machinery that predicts and avoids the stall. **A GPU hides it by switching to a different warp** (Module 02 §4).

Occupancy is the ratio of resident warps to the maximum the hardware supports per streaming multiprocessor.

More resident warps means more candidates to switch to when one stalls, so more of the latency is covered by useful work from other warps.

It is limited by three resources: registers per thread (registers are a fixed pool per SM, so using more per thread means fewer resident threads), shared memory per block (same argument), and block size (which must divide the limits sensibly). **Using more registers to avoid recomputation reduces occupancy**, which is a real and frequently non-obvious trade-off.

MAXIMUM OCCUPANCY IS NOT THE GOAL

Occupancy is a means, not an end. The actual goal is to saturate whichever resource limits the kernel — usually memory bandwidth (Module 07).

A kernel running at 50% occupancy that already saturates memory bandwidth is finished. Raising its occupancy to 100% achieves nothing whatsoever, because the bandwidth was already the constraint.

And more registers per thread can beat more warps. Instruction-level parallelism *within* a thread also hides latency — several independent loads in flight from one thread cover a stall just as effectively as switching warps does, and the registers that make it possible reduce occupancy.

Volkov's result is the canonical demonstration: several standard kernels run measurably *faster* at lower occupancy, with more work and more registers per thread. **Measure the kernel; do not maximise the metric** — which is a specific instance of a general rule worth carrying.

- **Check every CUDA error.** Kernel launches are asynchronous and report failure at the next synchronisation point — so without explicit checking you observe a wrong answer rather than an error, far from where it occurred. **This costs beginners more time than anything else;** wrap every call in a checking macro from the first line of code.
- **Use compute-sanitizer,** which provides out-of-bounds checking, race detection, and uninitialised memory detection for device code. The GPU equivalent of Module 03's ThreadSanitizer discipline.
- **Verify against a CPU implementation before optimising anything.** A fast wrong kernel is worth nothing, and optimisation frequently introduces subtle indexing errors.
- **Guard the bounds.** The grid size rounds up to a whole number of blocks, so the final block contains threads whose index exceeds the data. **if (i < n) is not optional,** and omitting it corrupts memory past the array.

- **Profile with Nsight Compute**, which reports the limiting resource, the achieved bandwidth, the occupancy, and the divergence directly — rather than requiring you to infer them.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
NVIDIA — CUDA C++ Programming Guide (free)	The authoritative reference for the execution and memory models of §1 and §2.
NVIDIA — CUDA C++ Best Practices Guide (free)	The practical companion — coalescing, occupancy, and the optimisation ordering. Shorter and more immediately useful.
OLCF — CUDA Training Series (free)	Structured exercises with solutions, paced sensibly, and they run on modest hardware.
Volkov — Better Performance at Lower Occupancy (free)	The §3 result. Short, measured, and it corrects a belief that is still widespread.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write, launch, and verify a vector addition kernel with full error checking and bounds guarding.
- 2 Omit the error checking, introduce a launch failure, and observe what you see instead of an error.
- 3 Measure the effect of block size on a simple kernel for sizes 32, 64, 128, 256, 512, 1024, and for 100 and 200.
- 4 **Measure divergence cost:** write a kernel whose branch depends on thread index modulo 2, and one where it depends on index divided by 32. Compare.
- 5 Measure a coalesced and a strided access pattern, sweeping the stride from 1 to 32. Plot the achieved bandwidth.
- 6 **Implement the array-of-structures case** and its structure-of-arrays equivalent, and report the measured ratio.
- 7 Use the occupancy calculator or Nsight to find your kernel's occupancy and what limits it.
- 8 Increase registers per thread deliberately and measure occupancy and performance together. Find a case where lower occupancy is faster.
- 9 Run compute-sanitizer on a kernel with a deliberate out-of-bounds access.
- 10 Place a kernel on the GPU's roofline (Module 07) and state what limits it.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Map thread, warp, block, and grid onto the hardware.
- 2 Why must blocks be independent, and what does that buy?
- 3 Why is the warp the real unit of execution, and what follows for block size?
- 4 What does a divergent branch cost within a warp, and between warps?
- 5 Name six GPU memory spaces with their scope and latency.
- 6 Why is 'local' memory a performance bug?
- 7 Explain coalescing and the cost of violating it.
- 8 What is occupancy for, and what three resources limit it?
- 9 Why is maximum occupancy not the goal?